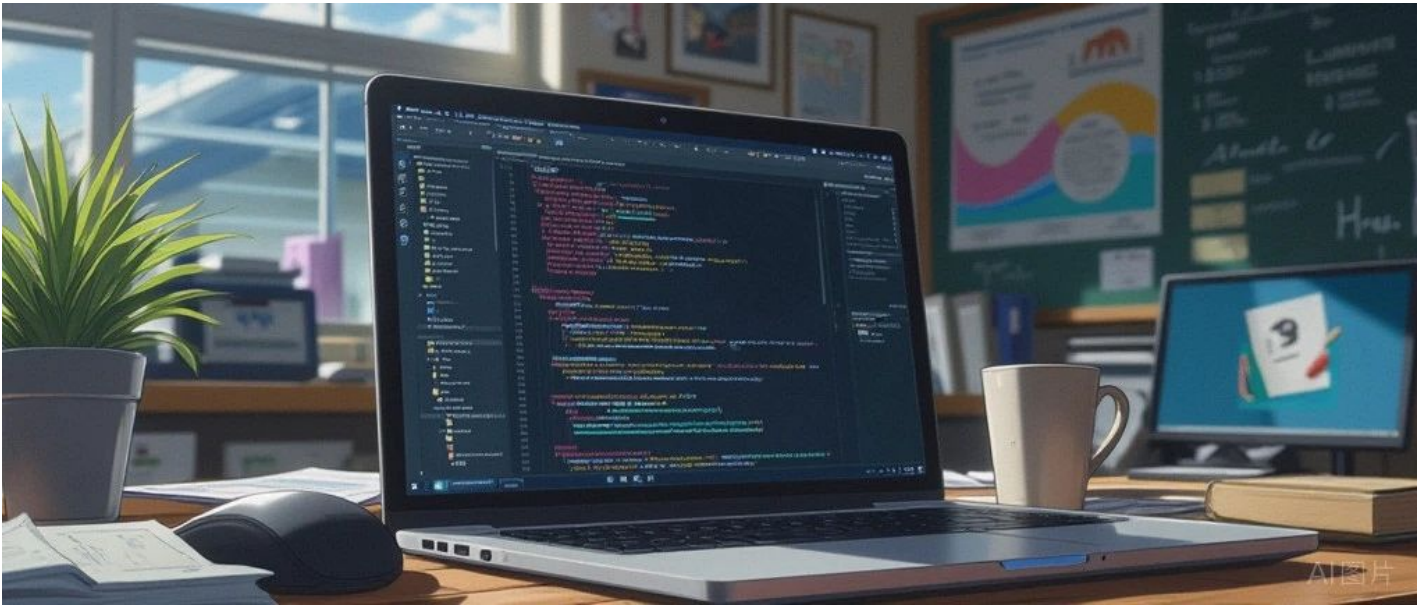




某招投标服务平台AST详解

原创 将往 土木佬的爬虫逆向日记

声明

本文逆向于2025年3月29日,仅做技术学习交流。本文中所有内容仅供学习交流使用,不用于其他任何目的,不提供完整代码,抓包内容、敏感网址、数据接口等均已做脱敏处理,严禁用于商业用途和非法用途,否则由此产生的一切后果均与作者无关!

一、前言

网站:

aHR0cHM6Ly9idWxsZXRpbi5jZWJwdWJzZXJ2aWNlLnNvbS94eGZiY21zZXMvc2VhcmNoL2J1bGxldGluLmhh0bWw/ZGF0ZXM9MzAwJmNhdmVnb3J5SWQ9ODgmcGFhZT0xJnNob3dTdGF0dXM9MSZ0aW11X18yNjUyPW40angyRGNEQmpEJTNEJTNEREtEc1RxQk1LN0kxS09XRfJtWU9ZRA==

适合AST学习

二、定位参数

抓包可以看到 time_2652 参数是加密的, 每一页都是不同的



time_2652

公众号 · 土木佬的爬虫逆向日记

不带这个参数的话会返回一个ob混淆的js。

通过调用栈可以定位到核心代码

```
V = m[lK(GF.m)](m[lK(GF.G)](m[lK(GF.k)](m[lK(GF.Q)](m[lK(GF.t)](L[lK(GF.b)](O), '|'), m[lK(GF.Y)](r)), '|'), ne  
w Date()[lK(GF.u)]()), '|1')  
O = F['ua'](V, !(-0x1c67 + 0x544 + 0x1723))
```

O即为 `time_2652` 参数，但是混淆的太多，不利于我们扣代码。因此需要先解混淆

三、还原数字数组

我们先看一下代码结构

```
function M() : [string, string, string, string, string, string, string, string, string, string, string, string] {  
    var GH : string[] = ['XK392UY', 'D8a/Qnx', 'q8ejD8Y', 'Rnc=yL9', 'Uez6Hi0GRUqnyT', '0Aq6XKf',  
    M = function() : string[] {...};  
    return M();  
}  
  
function e(l, m) {  
    var G : string[] = M();  
    return e = function(u, j) {...}  
    e(l, m);  
}  
  
(function(l, m : number) : void {  
    var lQ : {...} = [ 12 elements... ]  
    , l0 = e  
    , G : string[] = l();  
    while (!![]) {...}  
}(M, m: 0x565 * -0x86 + -0xa * -0x7267 + 0xf92d),  
!(function() : void {  
    var Gh : = [ 57 elements... ]  
    , GF : {...} = [ 13 elements... ]  
    , Gd : {...} = [ 25 elements... ]  
    , Gr : {...} = [ 22 elements... ]  
    , Ga : {...} = [ 7 elements... ]  
    , Go : {...} = [ 8 elements... ]  
    , Gg : {...} = [ 6 elements... ]  
    , Gu : {...} = [ 9 elements... ]  
    , GY : {...} = [ 28 elements... ]  
    , Gb : {...} = [ 6 elements... ]  
    , Gt : = [ 100 elements... ]  
    , GQ : {...} = [ 10 elements... ]
```

公众号 · 土木佬的爬虫逆向日记

代码结构

可以看到是由一个字符串大数组和许多十六进制小数组组成。

在加密代码中 `m[lK(GF.m)]`，由最里面的两个小数组在嵌套外面的大数组组成。因此我们先要还原 `lK(GF.m)` 两个十六进制小数组。

通过解析，可以看到都是 `VariableDeclarator` 类型初始化是 `ObjectExpression`

```

- declarations: [
  - VariableDeclarator {
    type: "VariableDeclarator"
    start: 7823
    end: 8871
    + loc: {start, end, filename, identifierName}
  - id: Identifier {
    type: "Identifier"
    start: 7823
    end: 7825
    + loc: {start, end, filename, identifierName}
    name: "Gh"
  }
  - init: ObjectExpression = $node {
    type: "ObjectExpression"
    start: 7828
    end: 8871
    + loc: {start, end, filename, identifierName}
    + properties: [57 elements]
  }
}

```

公众号 · 土木佬的爬虫逆向日记

解析

因此先用type类型判断下

```

traverse(ast, {
  VariableDeclarator(path) {
    if (types.isObjectExpression(path.node.init)) {
    }
  }
});

```

接着取id里面的name参数就是赋值参数，然后查找参数的绑定函数

```

let bingname = path.scope.getBinding(path.node.id.name);
let arrbd = bingname.referencePaths;
for (let i = 0; i < arrbd.length; i++) {
}

```

接下来找绑定函数的父对象。

```
arrbd[i].parentPath
```

即如下函数

```
354 let bingname = path.scope.getBinding(path.node.id.name);
355 let arrbd = bingname.referencePaths;
356 for (let i:number = 0; i < arrbd.length; i++) {
357     console.log(arrbd[i]+'');
358     let mempath:string = arrbd[i].parentPath;
359     console.log(mempath+'');
```

VariableDeclarator() > mempath

运行: astart.js

```
m
m[13(0x1de)]
m
m[14(0x208)]
m
m[15(0x1b2)]
m
m[15(0x234)]
m
m[15(0x26a)]
m
m[15(0x1e7)]
m
m[15(0x260)]
m
m[14(0x1d1)]
m
m[14(0x1c9)]
```

公众号 · 土木佬的爬虫逆向日记

函数

解析如下

(0(1Q.m)) / (0x1977 + -0xe

```
- arguments: [
  - MemberExpression {
    type: "MemberExpression"
    start: 6916
    end: 6920
    + loc: {start, end, filename, identifierName}
    + object: Identifier {type, start, end, loc, name}
    computed: false
  - property: Identifier {
    type: "Identifier"
    start: 6919
    end: 6920
    + loc: {start, end, filename, identifierName}
    name: "m"
  }
}
```



公众号·土木佬的爬虫逆向日记

解析

接下来判断下类型是否是 `MemberExpression`，并取其中的name参数

```
if (mempath.isMemberExpression()) {
  let proptype = mempath.node.property.name
}
```

我们现在有了值，接下来返回前面把数组我们保存下来

```

- init: ObjectExpression {
  type: "ObjectExpression"
  start: 6530
  end: 6752
+ loc: {start, end, filename, identifierName}
- properties: [
  - ObjectProperty {
    type: "ObjectProperty"
    start: 6540
    end: 6548
    + loc: {start, end, filename, identifierName}
    method: false
    + key: Identifier {type, start, end, loc, name}
    computed: false
    shorthand: false
    + value: NumericLiteral {type, start, end, loc, extra, ...
      +I}
  }
+ ObjectProperty {type, start, end, loc, method, ... +4}
+ ObjectProperty {type, start, end, loc, method, ... +4}
+ ObjectProperty {type, start, end, loc, method, ... +4}
+ ObjectProperty {type, start, end, loc, method, ... +4}
+ ObjectProperty {type, start, end, loc, method, ... +4}
+ ObjectProperty {type, start, end, loc, method, ... +4}
+ ObjectProperty {type, start, end, loc, method, ... +4}

```

公众号: 土木佬的爬虫逆向日记

数组

我们取 `init` 里面的 `properties` , 把 `key` 和 `value` 保存到一个数组里面。

```

let props = path.node.init.properties;
let obj = {};
for (let i = 0; i < props.length; i++) {
  obj[props[i].key.name] = props[i].value;
}

```

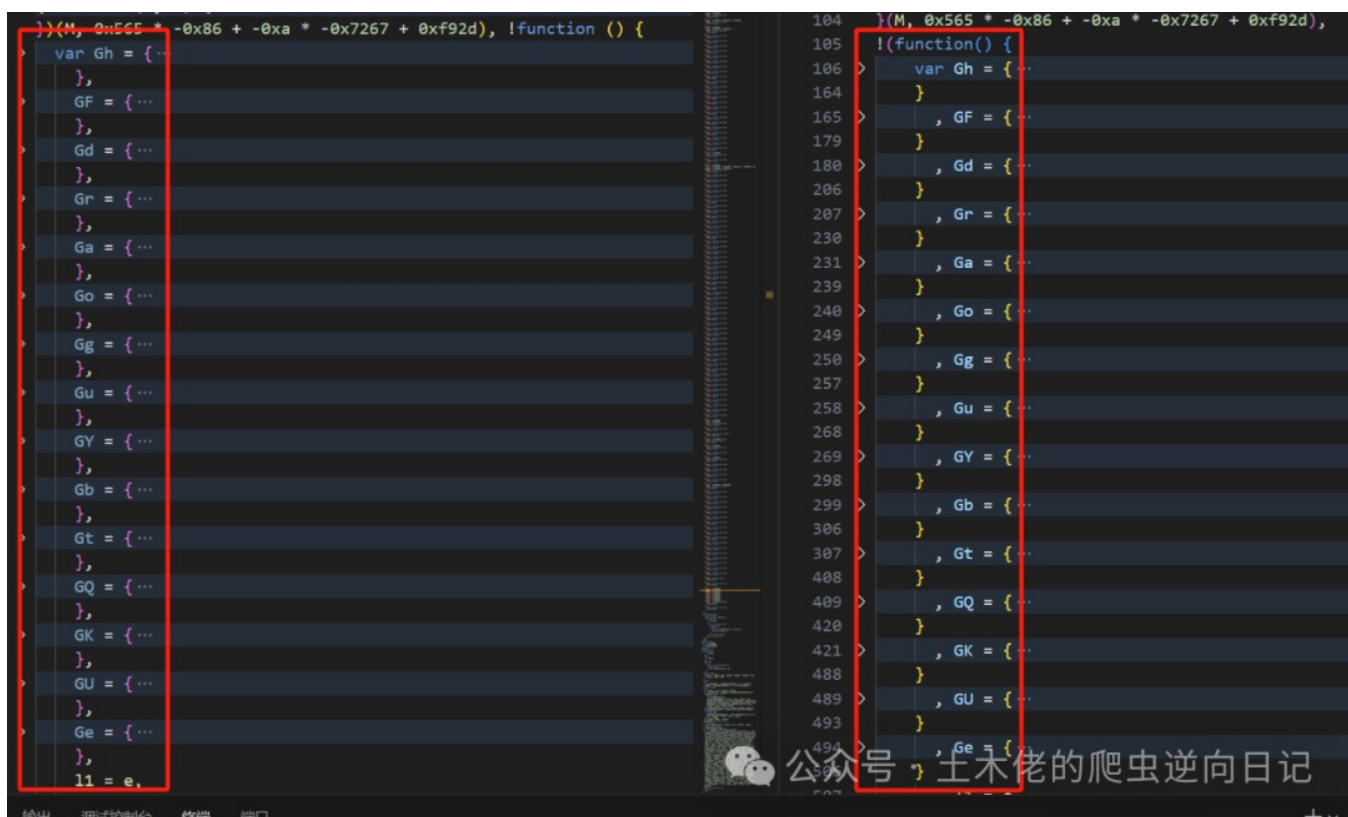
然后在下面匹配这个数组,并替换。

```

if (types.isNumericLiteral(obj[propname])) {
  mempath.replaceWith(obj[propname])
}

```

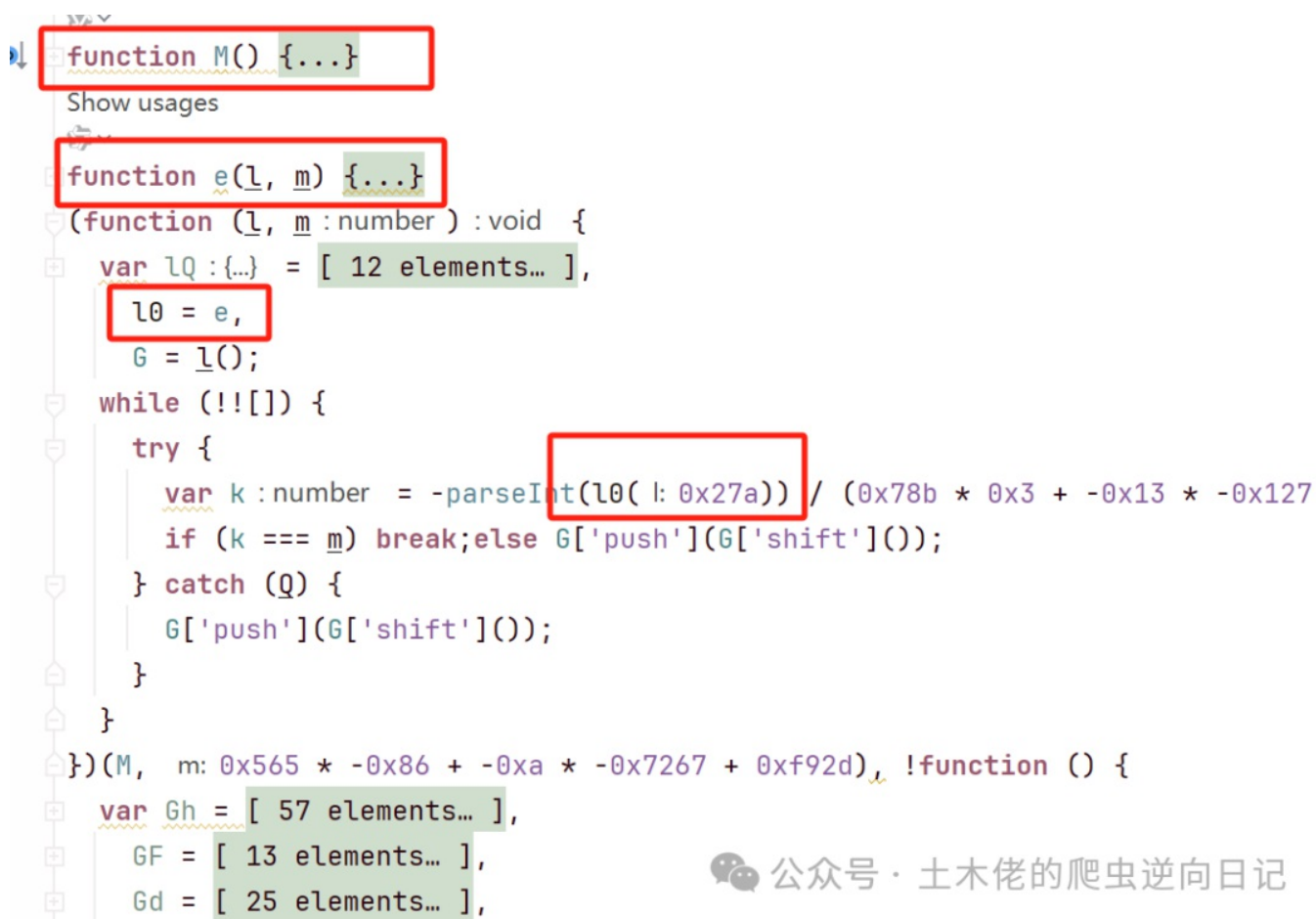
可以看到还原后的代码已经变灰, 没有调用了。



还原后

四、还原字符串数组

接下来我们还原第二层，字符串的数组还原



字符串的数组

可以看到如图，e赋值到l0调用。而e调用了M和e函数进行偏移解密。

先判断类型，这时我们要限定一下name是e

```
traverse(ast, {
  VariableDeclarator(path) {
    if (types.isIdentifier(path.node.init, {name: 'e'})) {
    }
  }
});
```

然后继续找他的绑定函数

```
let bingname = mempath.scope.getBinding(mempath.node.id.name);
let arrbd = bingname.referencePaths;
for (let i = 0; i < arrbd.length; i++) {
  let mempath = arrbd[i].parentPath;
}
```

接下来判断下是否是 `CallExpression` 类型，并取arguments

```
- CallExpression {
  type: "CallExpression"
  start: 6289
  end: 6298
+ loc: {start, end, filename, identifierName}
+ callee: Identifier {type, start, end, loc, name}
- arguments: [
  - NumericLiteral = $node {
    type: "NumericLiteral"
    start: 6292
    end: 6297
    + loc: {start, end, filename, identifierName}
    + extra: {rawValue, raw}
    value: 483
  }
]
}
```

公众号 · 土木佬的爬虫逆向日记

CallExpression

```
if (mempath.isCallExpression()) {
  let argu = mempath.get('arguments') + '';
}
```

我们把e函数剪切出来放到最上面，并且把检测函数去掉。

```

b['prototype']['LdIFdg'] = function() {
    var Y = new RegExp(this['JbuAMo'] + this['DNmxyA'])
    , u = Y['test'](this['UULPjk']['toString']()) ? --this['OEswPC'][0x75d*0x5+-0x17ee+-0xce2] :-
    -this['OEswPC'][0x26fb+0x191*-0x10+-0xdeb];
    return this['MxGFtc'](u);
}
//改为
b['prototype']['LdIFdg'] = function() {
    var Y = new RegExp(this['JbuAMo'] + this['DNmxyA'])
    , u = --this['OEswPC'][0x75d * 0x5 + -0x17ee + -0xce2];
    return this['MxGFtc'](u);
}

```

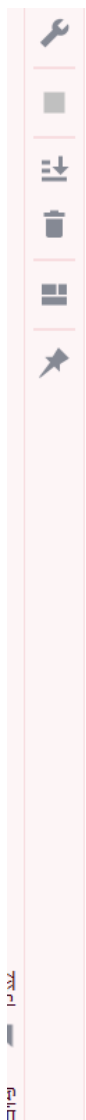
然后直接调用替换即可

```

let v1 = e(argu)
mempath.replaceWith(types.stringLiteral(v1))

```

最后可以看到混淆代码嵌套调用，因此要进行判断循环。



```

l0 = e
l1 = e
l5 = l4
l6 = l4
l7 = l4
l4 = l1
l9 = l8
l8 = l1
l1 = l1
l6 = lm
lm = l1
lM = l1
le = l1
lU = l1
lJ = l1
lc = l1
lk = lK
lK = l1
l2 = e
l3 = e

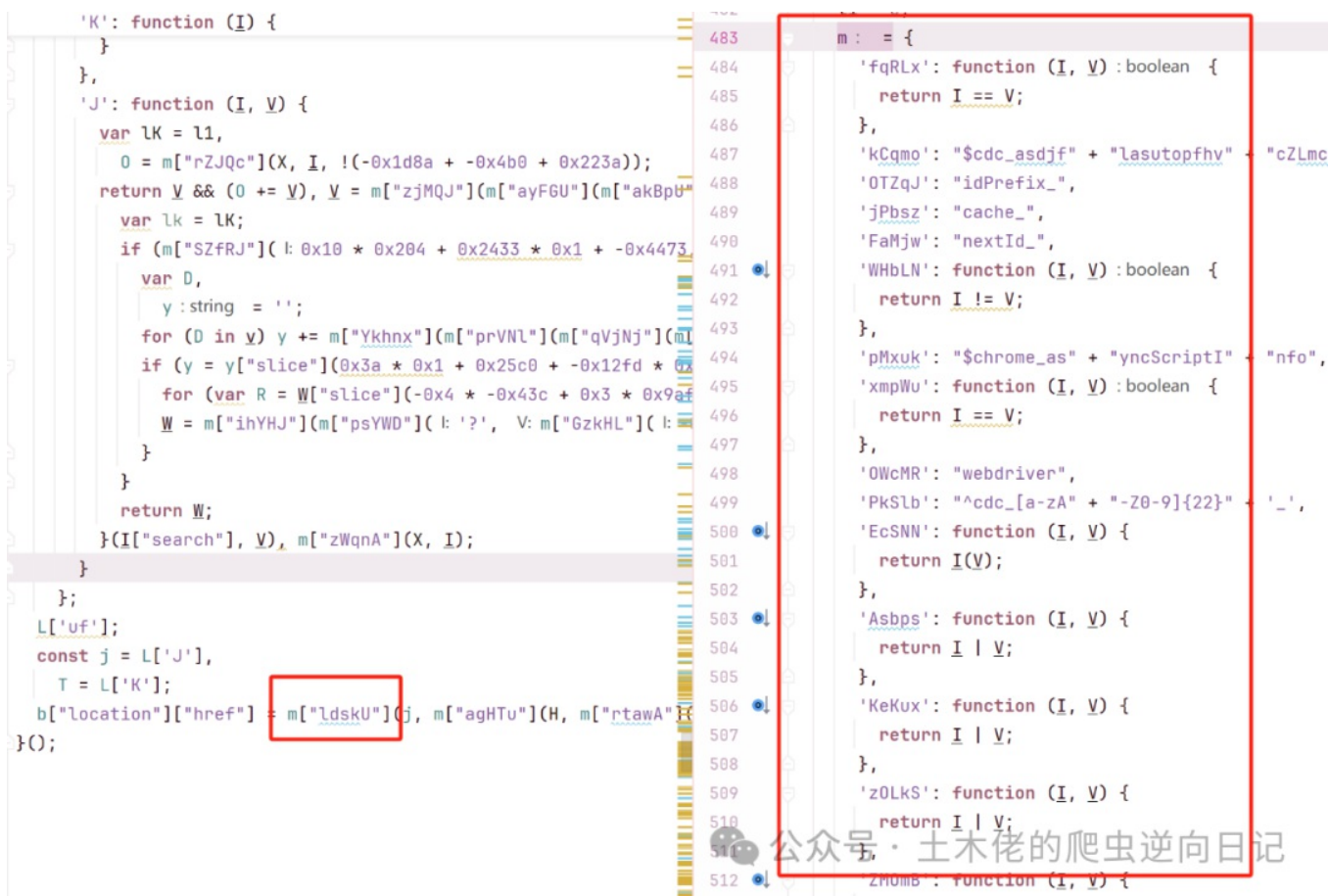
```

公众号 · 土木佬的爬虫逆向日记

嵌套

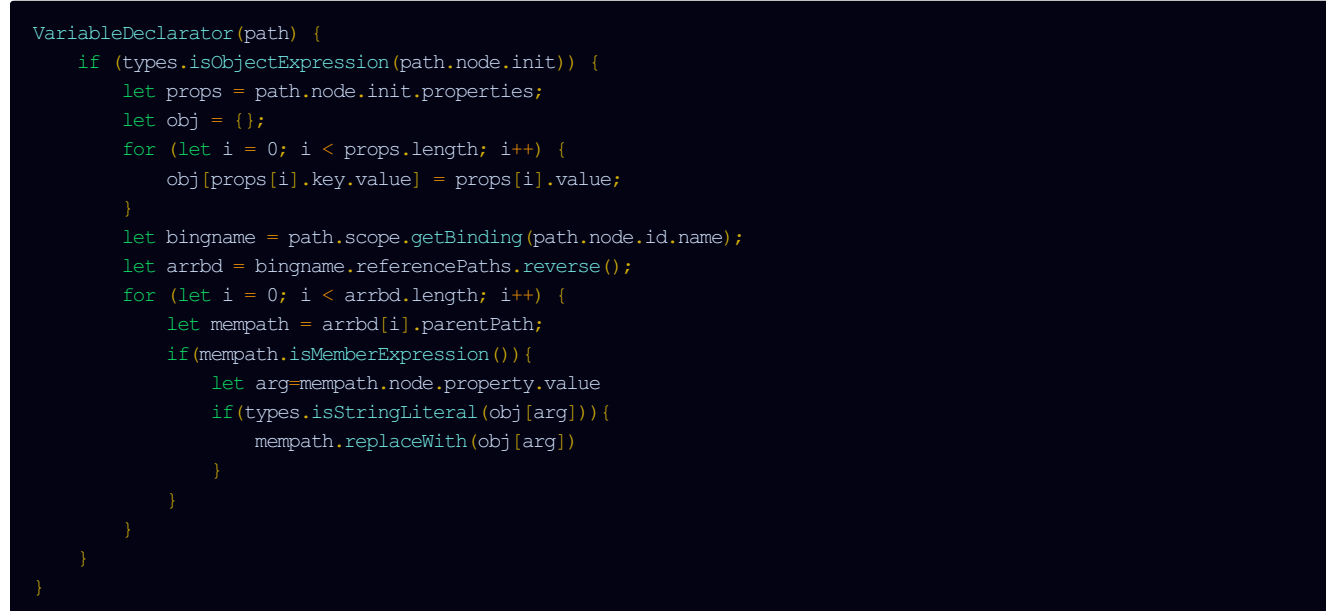
五、还原字符串大数组

接下来剩最后一个m数组。因为包含字符串和函数，我们最后处理。

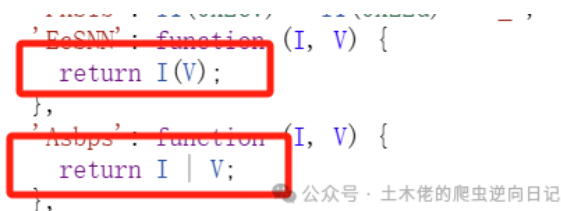


m数组

跟上面一样，先判断类型，在查找绑定，最后写入数组进行查询替换。



因为这个数组的 `FunctionExpression` 类型下有 `CallExpression` 和 `BinaryExpression` 两个类型。因此我们要分开处理



两种调用

继续在 `if(types.isStringLiteral(obj[arg]))` 下写判断。

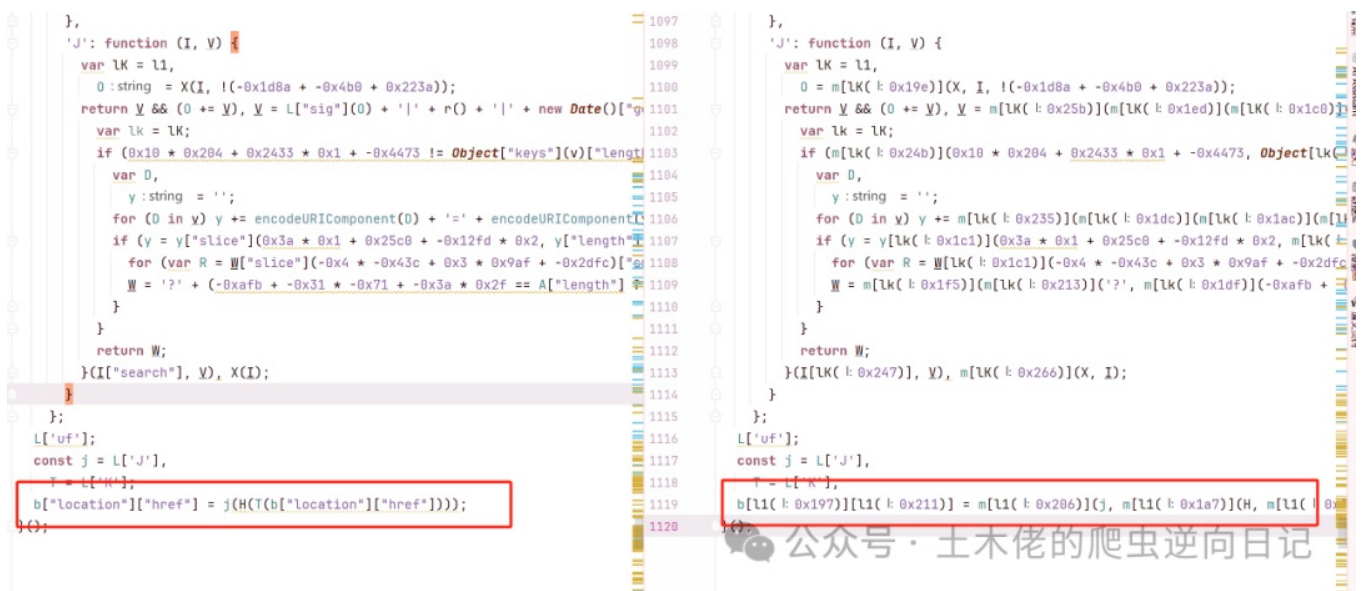
我们先判断是否是函数类型。接着取函数的body。然后对 `CallExpression` 和 `BinaryExpression` 两个类型进行判断。

当为 `CallExpression` 类型时取参数和调用，写入到原来的path替换

当为 `BinaryExpression` 类型时取left, right和operator，写入到原来的path替换

```
else if(types.isFunctionExpression(obj[arg])){
  let bd=obj[arg].body.body
  if(bd.length===1&&types.isReturnStatement(bd[0])){
    let argu=bd[0].argument;
    if(types.isCallExpression(argu)){
      let argus=mempath.parent.arguments;
      let calleename=argus[0];
      let newcall=types.callExpression(calleename,argus.slice(1))
      mempath.parentPath.replaceWith(newcall)
    }else if(types.isBinaryExpression(argu)){
      let argus=mempath.parent.arguments;
      let left=argus[0];
      let right=argus[1];
      let newcall=types.binaryExpression(argu.operator,left,right)
      mempath.parentPath.replaceWith(newcall)
    }
  }
}
```

至此我们就完成了大部分还原



还原

六、参数 time__2652 生成

还原后我们就可以很明显看到核心代码了

```
let data=sig(O) + '|' + r() + '|' + new Date() ["getTime"] () + '|1'
console.log(ua(data))
```

核心代码

只有46行

总结

网站难度不高。有兴趣的朋友可以试试。

github:

<https://github.com/jwmycz/WxArticleCode>

本文未经许可禁止转载，禁止任何修改后二次传播，擅自使用本文讲解的技术而导致的任何意外，作者均不负责，若有侵权，请在公众号联系作者立即删除！

个人观点，仅供参考